

# Лабораторная работа №8

Компьютерный практикум по статистическому анализу данных

---

Канева Екатерина, НФИбд-02-22

20 декабря 2025

Российский университет дружбы народов, Москва, Россия

## Информация

---

- Канева Екатерина Павловна
- студент группы НФИбд-02-22
- Российский университет дружбы народов
- 1132222004@rudn.ru
- <https://nevseros.github.io/ru/>

## Вводная часть

---

Основной целью работы является освоение специализированных пакетов Julia для решения задач оптимизации.

- Используя Jupyter Lab, повторить примеры.
- Выполнить задания для самостоятельной работы.

## Выполнение работы

---

## Примеры

---



# Примеры с линейным программированием

Выполнила примеры с линейным программированием:

```
[3]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer);

[4]: # Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0);

[5]: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 140)
@constraint(model, 7x + 12y >= 120);

[6]: # Определение целевой функции:
@objective(model, Min, 12x + 20y);

[7]: # Запуск функции оптимизации:
optimize!(model)

[8]: # Определение причины завершения работы оптимизатора:
termination_status(model)

[9]: OPTIMAL::TerminationStatusCode = 1

[9]: # Демонстрация полученных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результатов оптимизации:
@show objective_value(model);

value[x] = 14.599999999999999
value[y] = 1.25000000000000047
objective_value[model] = 205.0
```

Рис. 1: Примеры с линейным программированием.

# Примеры с векторизованными ограничениями и целевой функцией оптимизации

Выполнила примеры с векторизованными ограничениями и целевой функцией оптимизации:

```
[19]: # Определение обычной модели с именем vector_model:
vector_model = Model(gp.Optimizer);

[21]: # Определение начальных данных:
A = [ 1 1 9 5;
      1 5 0 8;
      2 0 6 12];
b = [7; 3; 5];
c = [1; 3; 5; 2];

[23]: # Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0);

[25]: # Определение ограничений модели:
@constraint(vector_model, A * x .== b);

[26]: # Определение целевой функции:
@objective(vector_model, Min, c' * x);

[29]: # Поиск функции оптимизации:
optimize!(vector_model)

[31]: # Определение причины завершения работы оптимизатора:
termination_status(vector_model)

[31]: OPTIMAL::TerminationStatusCode = 1

[33]: # Демонстрация результатов оптимизации:
@show objective_value(vector_model);
objective_value(vector_model) = 4.9238769238769225
```

Рис. 2: Примеры с векторизованными ограничениями и целевой функцией оптимизации.

# Пример по оптимизации рациона питания

Выполнила примеры по оптимизации рациона питания:

```
[30]: category_data = JuMP.Containers.DenseAxisArray{
      [1800 2200;
       91 Inf;
       0 55;
       0 1779],
      ["calories", "protein", "fat", "sodium"],
      ["min", "max"]};

[38]: # Массив данных с характеристиками продуктов:
      foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"];

[40]: # Массив стоимости продуктов:
      cost = JuMP.Containers.DenseAxisArray{
      [2.49, 2.89, 1.59, 1.89, 2.89, 1.99, 2.49, 0.89, 1.59],
      foods};

[42]: food_data = JuMP.Containers.DenseAxisArray{
      [410 24 26 730;
       620 12 10 1100;
       560 20 12 1000;
       380 4 19 270;
       320 12 10 930;
       320 15 12 830;
       520 31 12 1130;
       100 0 2.5 125;
       330 0 10 180],
      foods,
      ["calories", "protein", "fat", "sodium"]};

[44]: # Определение объекта модели с именем model:
      model = Model{GLPK.Optimizer};

[46]: # Определение массива:
      categories = ["calories", "protein", "fat", "sodium"];
```

Рис. 3: Примеры по оптимизации рациона питания.

# Пример по путешествию по миру

Выполнила пример по путешествию по миру:

```
[61]: # Подключаем пакеты:
import Plg
Plg.add("DelimitedFiles")
Plg.add("CSV")
using DelimitedFiles
using CSV;

Resolving package versions...
Project No packages added to or removed from 'C:\Users\ekaneva\.julia\environments\v1.12\Project.toml'
Manifest No packages added to or removed from 'C:\Users\ekaneva\.julia\environments\v1.12\Manifest.toml'
Resolving package versions...
Project No packages added to or removed from 'C:\Users\ekaneva\.julia\environments\v1.12\Project.toml'
Manifest No packages added to or removed from 'C:\Users\ekaneva\.julia\environments\v1.12\Manifest.toml'

[62]: # Скачиваем данные:
passportdata = readlm(joinpath("passport-index-dataset", "passport-index-matrix.csv"), "");

[63]: # Задать переменные:
cntr = passportdata[2:end,1]
vF = {x => typeof(x) == Int64 || x == "VF" || x == "VQA" ? 1 : 0}.(passportdata[2:end,2:end]);

[64]: # Определить объект модели с именем model:
model = Model{GLPK.Optimizer}();

[65]: # Переменные, ограничение и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum{ vF[i,j]*pass[i] for i in 1:length(cntr)} == 1)
@objective(model, Min, sum(pass));

[66]: # Вызвать функцию оптимизации:
JuMP.optimize!(model)
termination_status(model)

[66]: OPTIMAL::TerminationStatusCode = 1

[73]: # Просмотр результата:
print(JuMP.objective_value(model), " passports: ", join(cntr[findall(JuMP.value.(pass) .== 1)], ", ", ""))

34.0 passports: Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea-Bissau, Hong Kong, Iran, Kenya,
Kuwait, Liberia, Libya, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, North Korea, Palestine, Papua New Guinea, Qatar, Saudi Arabia, Sin
gapore, Somalia, South Sudan, Spain, Syria, Turkmenistan, United States
```

Рис. 4: Пример по путешествию по миру.

# Пример с портфельными инвестициями

Выполнила пример с портфельными инвестициями:



Рис. 5: Пример с портфельными инвестициями.

# Пример с восстановлением изображения

Выполнила пример с восстановлением изображения:

```
[107]: # Восстановление исходного изображения:
Kref = load("data/khiam-small.jpg");

[108]: K = copy(Kref)
p = prod(size(K))
missingIds = rand(1,p,400)
K[missingIds] = RGBColor(0.0,0.0,0.0)
K
Gray_K(K)

[109]: 
```

```
[110]: # Map into float64:
Y = Float64.(Gray_K);

[111]: correctIds = findall(Y[:, :, :] .!= 0)
X = Convex.Variable{size(Y)}
problem = minimize(nuclearnorm(X))
problem.constraints += X[correctIds] == Y[correctIds];

Warning: Concatenating collections of constraints together with '+' or '++' to produce a new list of constraints is deprecated. Instead, use 'vcat' to concatenate collections of constraints.
! @ Convex C:\Users\ekaneva\julia\packages\Convex\1f9d8\src\deprecations.jl:129

[112]: # Houghian program:
solve!(problem, SCS.Optimizer);
```

Рис. 6: Пример с восстановлением изображения.

## Задания для самостоятельного выполнения

---

# Задание 1

Выполнила первое задание для самостоятельной работы:

```
[120]: # Определение объекта модели с именем model:
model = Model(GLPK.Optimizer)

[121]: # Определение переменных x, y и граничных условий для них:
@variable(model, 0 <= x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0);

[123]: # Определение ограничений модели:
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10);

[124]: # Определение целевой функции:
@objective(model, Max, x1 + 2x2 + 5x3);

[125]: # Базовая функция оптимизации:
optimize!(model)
# Определение причины завершения работы оптимизатора:
termination_status(model)

[125]: OPTIMAL::TerminationStatusCode = 1

[130]: @show value(x1);
@show value(x2);
@show value(x3);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375
```

Рис. 7: Задание 1.



## Задание 2

Выполнила второе задание для самостоятельной работы:

```
[133]: # Определенные объекты модели с именем model:  
vector_model = Model(GLPK.Optimizer);  
  
[135]: A = [-1 1 3; 1 3 -7]  
b = [-5; 10]  
c = [1; 2; 5];  
  
[137]: @variable(vector_model, x[1:3] >= 0);  
  
[139]: # Определенные переменные модели:  
@constraint(vector_model, A * x .<= b)  
@constraint(vector_model, x[1] <= 10);  
  
[141]: # Определенная целевая функция:  
@objective(vector_model, Max, c' * x);  
  
[143]: # Вызов функции оптимизации:  
optimize!(vector_model)  
# Определенные причины завершения работы оптимизатора:  
termination_status(vector_model)  
  
[145]: OPTIMAL::TerminationStatusCode = 1  
  
[145]: # Демонстрация полученных результатов значений переменных x и y:  
@show value{x[1]};  
@show value{x[2]};  
@show value{x[3]};  
  
value{x[1]} = 19.0  
value{x[2]} = 2.1875  
value{x[3]} = 0.9375  
  
[147]: # Демонстрация результата оптимизации:  
@show objective_value(vector_model);  
  
objective_value(vector_model) = 19.0625
```

Рис. 8: Задание 2.

## Задание 3

Выполнила третье задание для самостоятельной работы:

```
[149]: using Random
[152]: m = 8
      n = 4
      Random.seed(1)
      A = rand(m, n)
      b = rand(n);

[154]: x = Variable(n);

[156]: objective = sum_squares(A*x - b)
      constraints = [x == 0];

[158]: problem = minimize(objective, constraints);

[159]: solve!(problem, GCS.Optimizer);

[ Info: [Convex.jl] Compilation finished: 5.27 seconds, 234.278 MiB of memory allocated
=====
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
=====
problem: variables = 5, constraints = 14
cones:   1: linear vars: 6
        2: soc vars: 10, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_feas: 1.0e-07
         alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
         max_iter: 100000, normalize: 1, rho_r: 1.00e-06
         acceleration_lookback: 10, acceleration_interval: 10
         compiled with swomp parallelization enabled
lin-sys:
  rz(4): 38, rz(P): 0

iter | pri res | dual res | gap | obj | scale | time (s)
=====
0 | 1.66e+01 | 1.80e+00 | 2.31e+01 | -1.14e+01 | 1.00e-01 | 7.04e-04
125 | 5.09e-06 | 2.11e-06 | 1.38e-05 | 6.66e-01 | 1.21e+00 | 2.22e-03
=====
status: solved
timings: total: 2.23e-03s = setup: 1.72e-04s + solve: 2.05e-03s
        lin-sys: 7.87e-05s, cones: 5.78e-05s, accel: 1.00e-05s
=====
objective = 0.665534
```

Рис. 9: Задание 3.

## Задание 4

Выполнила четвёртое задание для самостоятельной работы:

```
[163]: num_sections = 5
num_applications = 1000

min_capacity = [100, 100, 220, 100, 100]
max_capacity = [250, 250, 220, 250, 250]

Random.seed(2)
priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:num_applications]
priorities = hcat(priorities...)
priority_weights = sum(priorities, dims=1);

[164]: model = Model{GlpK.Optimizer};

[165]: @variables(model, begin
100 <= x1 <= 250
100 <= x2 <= 250
x3 == 220
100 <= x4 <= 250
100 <= x5 <= 250
end);

[166]: @constraint(model, x1 + x2 + x3 + x4 + x5 == 1000)
@objective(model, Min, x1 * priority_weights[1] +
x2 * priority_weights[2] +
x3 * priority_weights[3] +
x4 * priority_weights[4] +
x5 * priority_weights[5]);

[167]: optimize!(model)
termination_status(model)

[167]: OPTIMAL::TerminationStatusCode = 1

[168]: println("Результаты:")
println("x1: ", value(x1))
println("x2: ", value(x2))
println("x3: ", value(x3))
println("x4: ", value(x4))
println("x5: ", value(x5))
println("Оптимальное значение целевой функции: ", objective_value(model))

Результаты:
x1: 100.0
x2: 100.0
x3: 220.0
x4: 240.0
x5: 100.0
Оптимальное значение целевой функции: 5.99606192e9
```

Рис. 10: Задание 4.

## Задание 5

Выполнила пятое задание для самостоятельной работы:

```
[176]: coffee = ["Rat", "Cappuccino"]
      products = ["grains", "milk", "sugar"]

      cost = JuMP.Containers.DenseAxisArray{
        [400, 100],
        coffee
      }

      coffee_data = JuMP.Containers.DenseAxisArray{
        [40 140 5;
         30 120 0],
        coffee,
        products
      }

      store = JuMP.Containers.DenseAxisArray{[500, 2000, 40], products};

[178]: model = Model{GLPK.Optimizer};

[180]: @variables(model, begin
      buy[coffee] == 0
    end)
      @objective(model, Max, sum(cost[i] * buy[i] for i in coffee))
      @constraint(model, [p in products], sum(coffee_data[i, p] * buy[i] for i in coffee) <= store[p])
      @constraint(model, coffee_data["Rat", "sugar"] * buy["Rat"] == 40);

[181]: JuMP.optimize!(model)
      term_status = JuMP.termination_status(model)

[181]: OPTIMAL::TerminationStatusCode = 1

[184]: hcat(buy.data, JuMP.value.(buy.data))

[184]: 2x2 Matrix{AffExpr}:
      buy[Rat]      0
      buy[Cappuccino] 6
```

Рис. 11: Задание 5.

## Заключение

---

Освоила пакеты Julia для решения задач оптимизации.